Devixo Solutions

AI/ML Internship Program

Task 02

Build and Compare Machine Learning Models

Project: Customer Churn Prediction

Student Name: Aman Hameed Ullah

Internship Domain: AI/ML

Submission Date: 01-11-2026

Table of Contents

SECTION 1: Introduction 3
SECTION 2: Objective 3
SECTION 3: Dataset Description 4
SECTION 4: Methodology 4
SECTION 5: Data Preprocessing 5
SECTION 6: Model Development 8
SECTION 7: Model Evaluation 10
SECTION 8: Regression Task 13
SECTION 9: Model Comparison 14
SECTION 10: Hyperparameter Tuning (Bonus) 15
SECTION 11: Best Model Analysis 16
SECTION 12: Conclusion 18
SECTION 13: GitHub Repository Link 18

SECTION 1: Introduction

Customer churn means that a customer stops using a company's service. In the telecom industry, winning a new customer costs much more than keeping an existing one, so companies want to know in advance which customers are likely to leave and offer them a reason to stay. Machine learning can learn this from past customer data.

This report documents Task 02 of the Devixo Solutions AI/ML Internship Program. Several machine learning models were built on the Telco Customer Churn dataset, evaluated with suitable metrics, compared, tuned with hyperparameter search and the best model was recommended. The report follows the five parts of the task: data preprocessing, model development, model evaluation, model comparison and conclusion, plus the bonus task of hyperparameter tuning.

SECTION 2: Objective

The objectives of this project are to:

- Select a real-world Kaggle dataset with at least 1,000 records and prepare it for machine learning.
- Train at least three different algorithms and evaluate each with appropriate metrics.
- Compare the models by training time, metrics, advantages and limitations.
- Recommend the best-performing model and explain why it performed better.
- Improve the models with hyperparameter tuning (GridSearchCV and RandomizedSearchCV).

Table 1: Task requirements and where they are covered

Task Requirement	Covered in
Part 1: Data preprocessing (load, missing values, duplicates, encoding, scaling, split)	Section 5
Part 2: Model development (6 classification models, Linear Regression)	Sections 6 and 8
Part 3: Evaluation (accuracy, precision, recall, F1, confusion matrix, MAE, MSE, RMSE, R^2)	Sections 7 and 8
Part 4: Model comparison (training time, metrics, advantages, limitations)	Section 9
Part 5: Conclusion and best model	Section 12
Bonus: Hyperparameter tuning	Section 10

SECTION 3: Dataset Description

Table 2: Dataset details

Item	Detail
Dataset Name	Telco Customer Churn
Source	Kaggle: https://www.kaggle.com/datasets/blastchar/telco-customer-churn
Creator	BlastChar (IBM sample data)
Size	7,043 customers × 21 columns
File	telco_churn.csv
Target Column	Churn (Yes / No): 26.54% of customers churned
Problem Type	Binary classification (churn); regression (MonthlyCharges) as an additional task

Each row describes one customer: demographic information, the services the customer uses, contract and billing details, and whether the customer left the company. The dataset contains numerical and categorical columns, which makes it suitable for comparing several algorithms. The columns are grouped in Table 3.

Table 3: Column groups of the dataset

Group	Columns	Description
Customer	gender, SeniorCitizen, Partner, Dependents, tenure	Demographics and months with the company
Services	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies	Services the customer has subscribed to
Account	Contract, PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges	Contract type and billing details
Target	Churn	Whether the customer left (Yes) or stayed (No)

SECTION 4: Methodology

The project followed this workflow:

Data Loading → Preprocessing → Model Development → Evaluation → Comparison → Hyperparameter Tuning → Conclusion

Tools and technologies used:

- Programming language: Python 3
- Environment: Google Colab (Jupyter notebook)

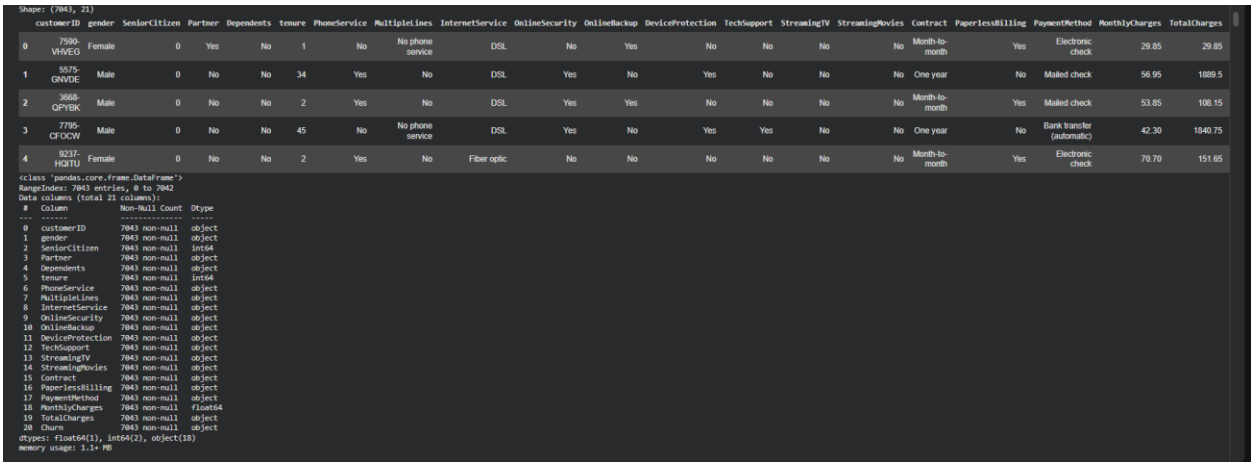
- Libraries: Pandas and NumPy (data handling), Matplotlib and Seaborn (charts), Scikit-learn (preprocessing, models and metrics), XGBoost (boosting models)
- Storage and version control: Google Drive and GitHub

SECTION 5: Data Preprocessing

Preprocessing prepares the raw data so that the models can learn from it correctly. All steps were done in the order below, and the summary is given in Table 5.

5.1 Loading the Data and Understanding the Target

The dataset was loaded from Google Drive with pd.read_csv(). It has 7,043 rows and 21 columns. The column TotalCharges is stored as text (object) although it should be numeric, which is the first sign of a hidden problem. About 73.46% of the customers stayed and 26.54% left, so the classes are imbalanced: a model that always predicts "No churn" would already reach about 73% accuracy. For this reason precision, recall and F1 score are used together with accuracy.



Shape: (7043, 21)

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	Yes	No	No	No	No	Month-to-month	Yes	Electronic check	29.85	29.85
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	No	Yes	No	No	No	One year	No	Mailed check	56.95	1889.5
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	Yes	No	No	No	No	Month-to-month	Yes	Mailed check	53.85	108.15
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	No	Yes	Yes	No	No	One year	No	Bank transfer (automatic)	42.30	1840.75
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	No	No	No	No	No	Month-to-month	Yes	Electronic check	70.70	151.65

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):

#	Column	Non-Null Count	Dtype
0	customerID	7043 non-null	object
1	gender	7043 non-null	object
2	SeniorCitizen	7043 non-null	int64
3	Partner	7043 non-null	object
4	Dependents	7043 non-null	object
5	tenure	7043 non-null	int64
6	PhoneService	7043 non-null	object
7	MultipleLines	7043 non-null	object
8	InternetService	7043 non-null	object
9	OnlineSecurity	7043 non-null	object
10	OnlineBackup	7043 non-null	object
11	DeviceProtection	7043 non-null	object
12	TechSupport	7043 non-null	object
13	StreamingTV	7043 non-null	object
14	StreamingMovies	7043 non-null	object
15	Contract	7043 non-null	object
16	PaperlessBilling	7043 non-null	object
17	PaymentMethod	7043 non-null	object
18	MonthlyCharges	7043 non-null	float64
19	TotalCharges	7043 non-null	object
20	Churn	7043 non-null	object

dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

Figure 1: Dataset records and information

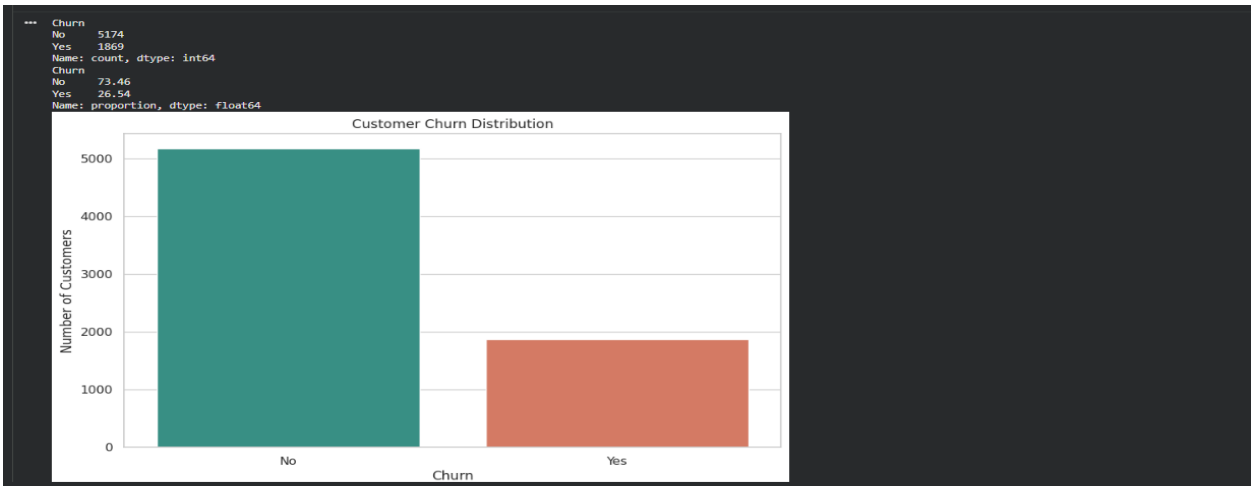
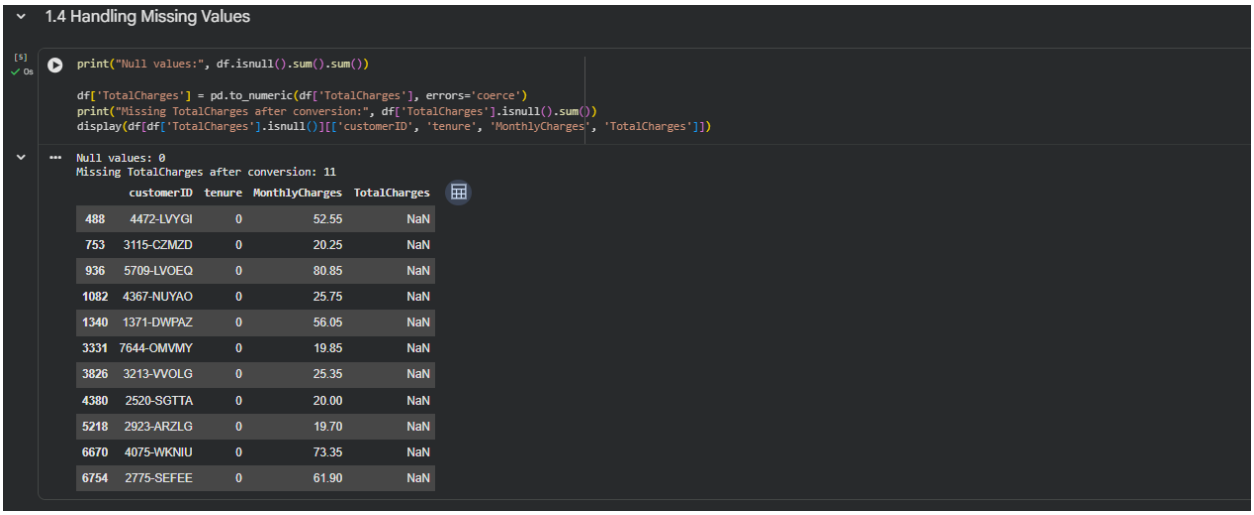


Figure 2: Distribution of the target variable (Churn)

5.2 Handling Missing Values

Pandas first reported no missing values, but TotalCharges contained blank spaces. After converting the column to numeric, 11 values became missing. All 11 customers have tenure 0, which means they are new customers who have not been billed yet. The values were therefore filled with 0 instead of the mean or median, because a customer with no months of service has logically paid nothing so far.



1.4 Handling Missing Values

```
print("Null values:", df.isnull().sum().sum())

df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
print("Missing TotalCharges after conversion:", df['TotalCharges'].isnull().sum())
display(df[df['TotalCharges'].isnull()][['customerID', 'tenure', 'MonthlyCharges', 'TotalCharges']])
```

```
Null values: 0
Missing TotalCharges after conversion: 11
```

	customerID	tenure	MonthlyCharges	TotalCharges
488	4472-LVYGI	0	52.55	NaN
753	3115-CZMZD	0	20.25	NaN
936	5709-LVOEQ	0	80.85	NaN
1082	4367-NUYAO	0	25.75	NaN
1340	1371-DWPAZ	0	56.05	NaN
3331	7644-OMVMY	0	19.85	NaN
3826	3213-VVOLG	0	25.35	NaN
4380	2520-SGTTA	0	20.00	NaN
5218	2923-ARZLG	0	19.70	NaN
6670	4075-WKNIU	0	73.35	NaN
6754	2775-SEFEE	0	61.90	NaN

Figure 3: Customers with missing TotalCharges (all with tenure 0)

5.3 Duplicates and Unnecessary Columns

No duplicate rows were found and every customerID is unique, so each row is a different customer and no rows were removed. The column customerID was dropped because it is only an identifier and carries no information about behavior; keeping it could make a model memorize individual customers.

5.4 Feature Engineering

Two new features were created from existing columns (Table 4) to give the models information that is not directly visible.

Table 4: Engineered features

Feature	Definition	Reason
num_services	Number of extra services used (OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies)	Customers with more services may be more attached to the company
is_auto_pay	1 if the payment method is automatic (bank transfer or credit card), otherwise 0	Automatic payment may make customers less likely to leave

5.5 Encoding Categorical Features

Machine learning models need numbers. Churn, gender, Partner, Dependents, PhoneService and PaperlessBilling have two categories and were converted to 0 and 1. The remaining text columns (such as Contract, InternetService and PaymentMethod) were converted with one-hot encoding, which creates a separate 0/1 column for each category; drop_first=True removes one column per feature to avoid duplicated information. After encoding, the dataset has 33 columns (32 features and the target).

5.6 Train-Test Split

The data was split into 80% for training (5,634 customers) and 20% for testing (1,409 customers). The models learn from the training set and are evaluated on the test set, which they have never seen. stratify=y keeps the churn ratio at 26.54% in both sets, and random_state=42 makes the split reproducible.

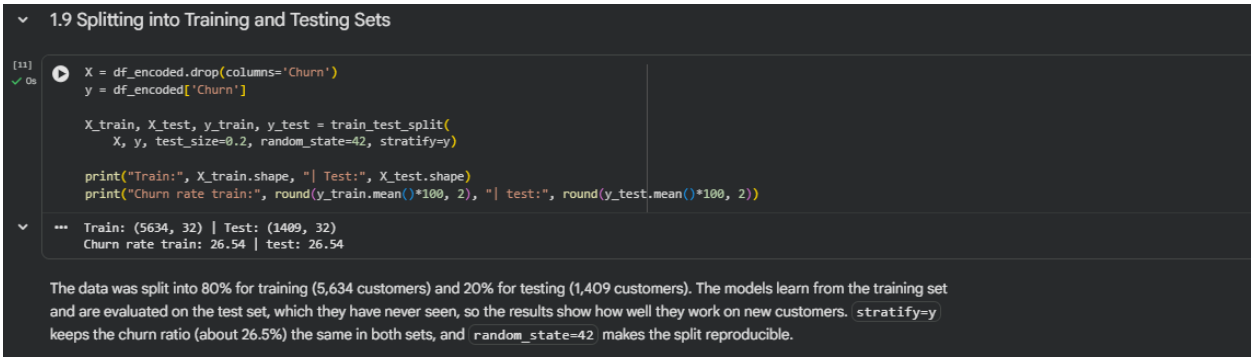


Figure 4: Train-test split output

5.7 Scaling Numerical Features

The numerical columns (tenure, MonthlyCharges, TotalCharges and num_services) were standardized to mean 0 and standard deviation 1. Scaling is important for KNN, SVM and Logistic Regression, because a large-valued column such as TotalCharges (up to about 8,600) would otherwise dominate small-valued columns such as tenure. Tree-based models are not affected by scaling.

The scaler was fitted only on the training data and then applied to the test data. Fitting it on the full dataset would leak information from the test set into training (data leakage) and make the results look better than they are. For this reason the split was done before scaling, which is slightly different from the order in which the task lists these steps.

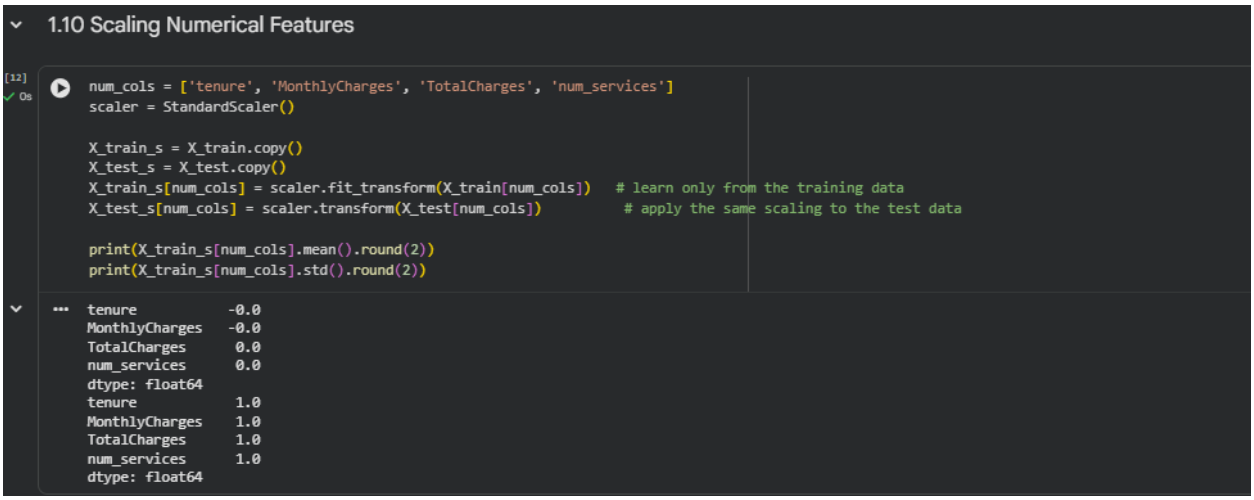


Figure 5: Scaling output (mean 0, standard deviation 1)

Table 5: Summary of preprocessing steps

Step	Problem	Action	Reason
Missing values	TotalCharges stored as text, 11 blanks	Converted to numeric, filled with 0	New customers with tenure 0
Duplicates	None found	No rows removed	Each customerID is unique
Identifier	customerID has no predictive value	Dropped	Avoid memorization
Feature engineering	Information spread over columns	Created num_services, is_auto_pay	Extra signals for models
Encoding	Models need numbers	0/1 mapping and one-hot encoding	32 numeric features
Imbalance	26.54% churn	Stratified split, F1 and recall used	Fair evaluation
Scaling	Different value ranges	StandardScaler fitted on training data only	Avoid data leakage

SECTION 6: Model Development

Six classification models from different algorithm families were trained to predict churn: Logistic Regression, Decision Tree, Random Forest, KNN, SVM and XGBoost (optional). Models that depend on distances or weights (Logistic Regression, KNN, SVM) used the scaled data, while tree-based models used the original data because scaling does not change them.

Table 6: Classification models used

Model	Family	How it works	Data Used
Logistic Regression	Linear	Combines features with weights and converts the result to a probability with the sigmoid function	Scaled

Model	Family	How it works	Data Used
Decision Tree	Tree	Splits the data with yes/no questions until it reaches a prediction (no depth limit)	Original
Random Forest	Ensemble (bagging)	Combines the votes of 100 trees trained on random samples of rows and features	Original
KNN	Distance-based	Predicts the most common class among the 5 most similar customers	Scaled
SVM	Kernel-based	Finds the boundary with the largest margin between classes (RBF kernel)	Scaled
XGBoost	Boosting	Builds trees one after another, each correcting the errors of the previous ones	Original

A helper function trained each model, measured the training time and calculated training and testing accuracy. Comparing the two accuracies shows whether a model overfits. The results are given in Table 7.

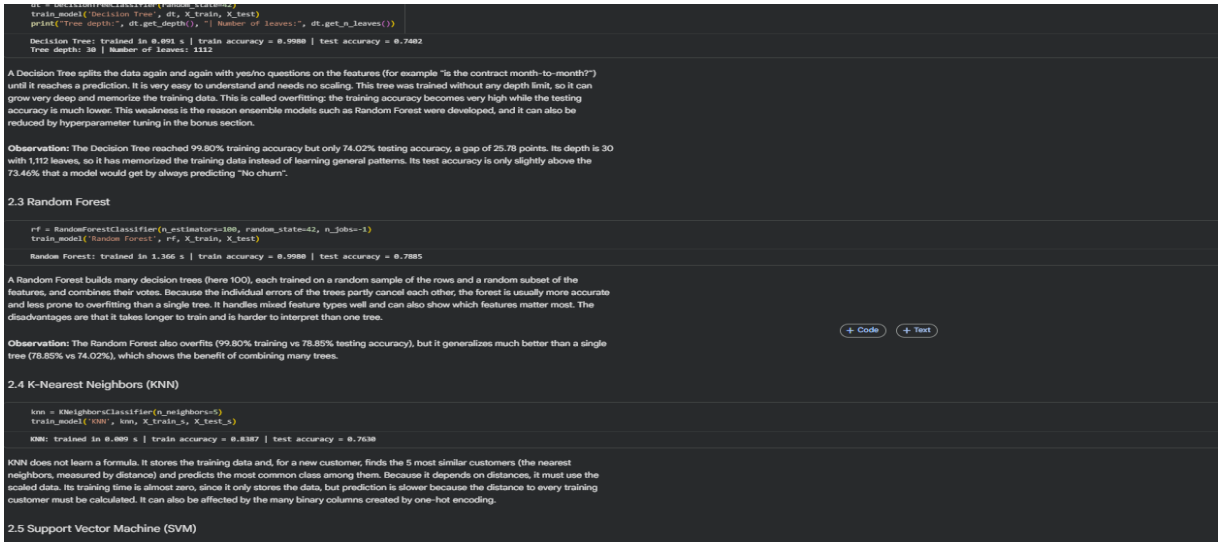


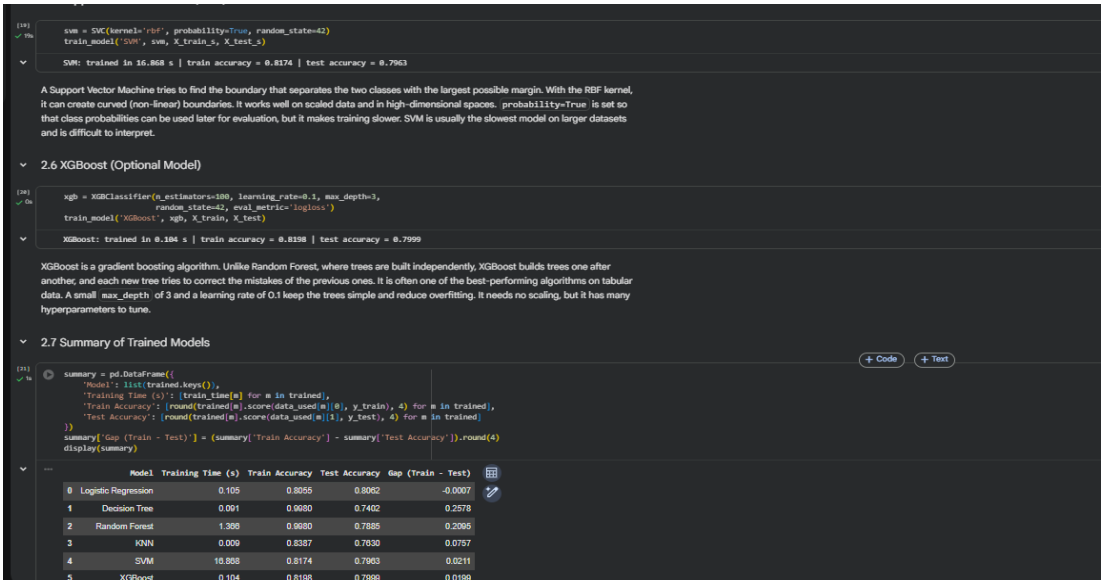
Figure 6: Code and output of model training

Table 7: Training time and accuracy of the models

Model	Time (s)	Train Acc.	Test Acc.	Gap
Logistic Regression	0.126	80.55%	80.62%	-0.07
Decision Tree	0.083	99.80%	74.02%	25.78
Random Forest	1.770	99.80%	78.85%	20.95
KNN	0.013	83.87%	76.30%	7.57
SVM	9.481	81.74%	79.63%	2.11
XGBoost	0.136	81.98%	79.99%	1.99

Logistic Regression has the highest testing accuracy and almost no gap between training and testing accuracy. The Decision Tree (depth 30, 1,112 leaves) and the Random Forest reach 99.80%

on training data but much less on test data, so they overfit strongly. The test accuracy of the Decision Tree (74.02%) is only slightly above the 73.46% baseline. XGBoost and SVM generalize well, but SVM is by far the slowest model. Training times vary slightly from run to run.



```
svm = SVC(kernel='rbf', probability=True, random_state=42)
train_model('SVM', svm, X_train_s, X_test_s)
```

```
SVM: trained in 16.868 s | train accuracy = 0.8174 | test accuracy = 0.7963
```

A Support Vector Machine tries to find the boundary that separates the two classes with the largest possible margin. With the RBF kernel, it can create curved (non-linear) boundaries. It works well on scaled data and in high-dimensional spaces. `probability=True` is set so that class probabilities can be used later for evaluation, but it makes training slower. SVM is usually the slowest model on larger datasets and is difficult to interpret.

2.6 XGBoost (Optional Model)

```
xgb = XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=3,
                    random_state=42, eval_metric='logloss')
train_model('XGBoost', xgb, X_train, X_test)
```

```
XGBoost: trained in 0.104 s | train accuracy = 0.8198 | test accuracy = 0.7999
```

XGBoost is a gradient boosting algorithm. Unlike Random Forest, where trees are built independently, XGBoost builds trees one after another, and each new tree tries to correct the mistakes of the previous ones. It is often one of the best-performing algorithms on tabular data. A small `max_depth` of 3 and a learning rate of 0.1 keep the trees simple and reduce overfitting. It needs no scaling, but it has many hyperparameters to tune.

2.7 Summary of Trained Models

+ Code + Text

```
summary = pd.DataFrame({
    'Model': list(trained.keys()),
    'Training Time (s)': [train_time[m] for m in trained],
    'Train Accuracy': [round(trained[m].score(data_used[m][0], y_train), 4) for m in trained],
    'Test Accuracy': [round(trained[m].score(data_used[m][1], y_test), 4) for m in trained]
})
summary['Gap (Train - Test)'] = (summary['Train Accuracy'] - summary['Test Accuracy']).round(4)
display(summary)
```

	Model	Training Time (s)	Train Accuracy	Test Accuracy	Gap (Train - Test)
0	Logistic Regression	0.105	0.8055	0.8062	-0.0007
1	Decision Tree	0.091	0.9080	0.7402	0.2578
2	Random Forest	1.386	0.9080	0.7885	0.2095
3	KNN	0.009	0.8387	0.7630	0.0757
4	SVM	16.868	0.8174	0.7963	0.0211
5	XGBoost	0.104	0.8198	0.7999	0.0199

Figure 7: Summary of trained models

SECTION 7: Model Evaluation

7.1 Evaluation Metrics

The models were evaluated on the test set of 1,409 customers. The positive class is "Churn = Yes".

- Accuracy: correct predictions divided by all predictions. It is misleading on imbalanced data.
- Precision = TP / (TP + FP): of the customers predicted to churn, how many really churned.
- Recall = TP / (TP + FN): of the customers who really churned, how many the model found.
- F1 Score: the harmonic mean of precision and recall.
- Confusion Matrix: counts of true negatives (TN), false positives (FP), false negatives (FN) and true positives (TP).
- ROC-AUC: how well the model ranks churners above non-churners (1.0 is perfect, 0.5 is random).

For churn prediction, recall is especially important, because losing a customer is more costly than sending a retention offer to a customer who would have stayed.

7.2 Classification Results

Table 8: Classification metrics of the models with default settings

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.8062	0.6593	0.5588	0.6049	0.8422
XGBoost	0.7999	0.6544	0.5214	0.5804	0.8442
SVM	0.7963	0.6537	0.4947	0.5632	0.7955
Random Forest	0.7885	0.6275	0.5000	0.5565	0.8217
KNN	0.7630	0.5565	0.5267	0.5412	0.7783
Decision Tree	0.7402	0.5109	0.5027	0.5067	0.6641

Logistic Regression is the best model on accuracy (80.62%), precision (65.93%), recall (55.88%) and F1 score (60.49%). XGBoost has the highest ROC-AUC (0.8442), only slightly above Logistic Regression (0.8422). The Decision Tree is the weakest model (F1 0.5067, ROC-AUC 0.6641). Even the best recall is only 55.88%, so about 44% of the churners are still missed at the default threshold of 0.5.

7.3 Confusion Matrices

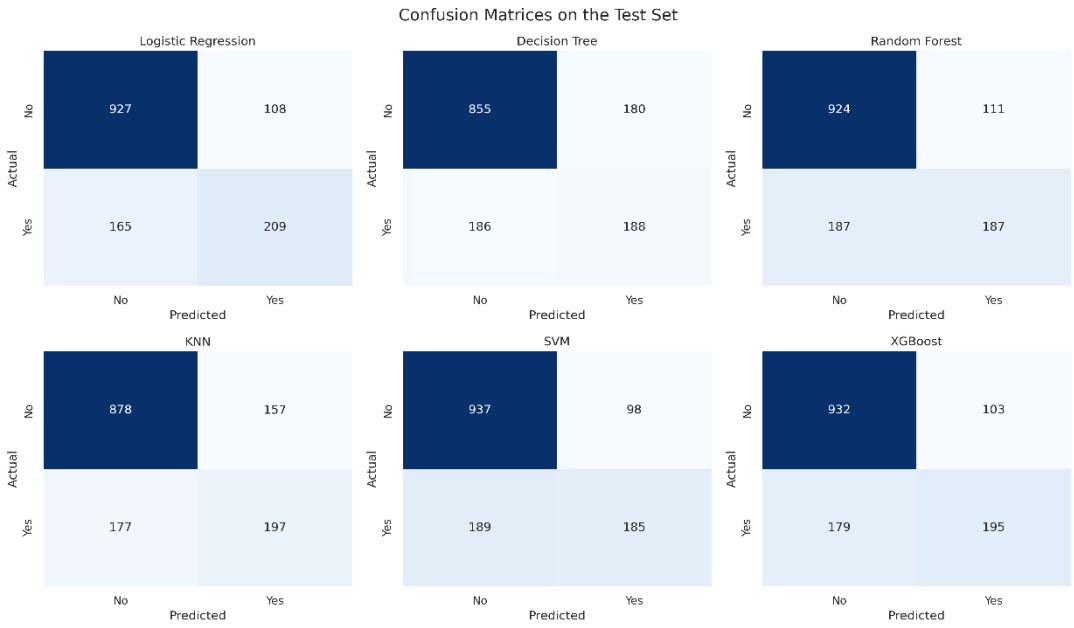


Figure 8: Confusion matrices of the six models on the test set

Table 9: Confusion matrix counts on the test set

Model	TN	FP	FN	TP
Logistic Regression	927	108	165	209
XGBoost	932	103	179	195
SVM	937	98	189	185

Model	TN	FP	FN	TP
Random Forest	924	111	187	187
KNN	878	157	177	197
Decision Tree	855	180	186	188

The test set contains 374 customers who actually churned. Logistic Regression finds 209 of them and misses 165, while SVM finds only 185 and misses 189. SVM makes the fewest false alarms (98), but it pays for this with the most missed churners. Since a lost customer is more costly than an unnecessary offer, models with fewer false negatives are preferred.

7.4 Metrics Comparison and Overfitting Check

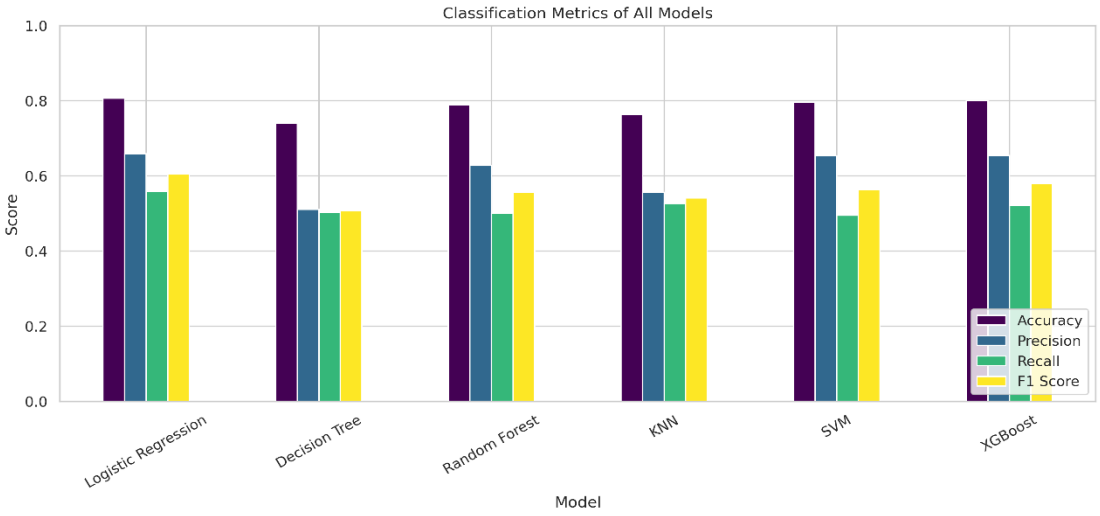


Figure 9: Classification metrics of all models

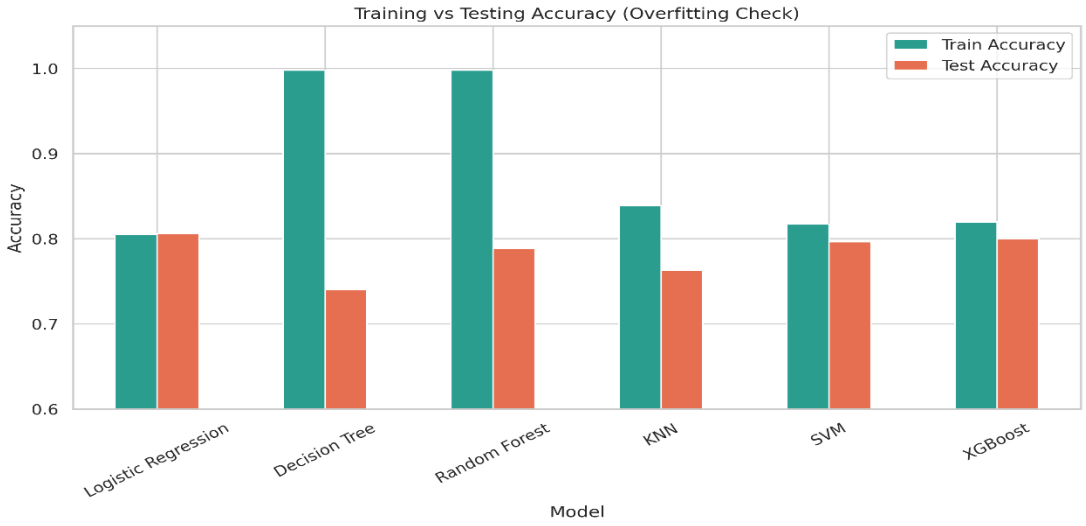


Figure 10: Training vs testing accuracy (overfitting check)

Logistic Regression has the highest bars on all four metrics, and recall and F1 are clearly lower than accuracy for every model, which confirms that accuracy alone hides the weak detection of churners. The second chart shows that Logistic Regression, SVM and XGBoost have nearly equal training and testing accuracy, while the Decision Tree and Random Forest score almost 100% on training data but much lower on test data.

SECTION 8: Regression Task

Churn is a classification problem, so Linear Regression and the regression metrics (MAE, MSE, RMSE and R^2) do not apply to it. To cover them, a second task was added: predicting the continuous value MonthlyCharges (the monthly bill, mean 64.76, from 18.25 to 118.75). TotalCharges was removed from the features because it is almost tenure × MonthlyCharges and would give away the answer (data leakage), and Churn was removed because it is not a pricing feature. The data was split 80/20 and four models were trained.

MAE is the average error in dollars, MSE is the average squared error, RMSE is the square root of MSE in dollars, and R^2 is the share of the variation in the bill that the model explains (1.0 is perfect).

Table 10: Regression results for MonthlyCharges

Model	Time (s)	MAE	MSE	RMSE	R^2 Score
Linear Regression	0.025	0.7893	1.1052	1.0513	0.9988
XGBoost	1.028	0.8193	1.1977	1.0944	0.9987
Random Forest	2.520	0.8966	1.4211	1.1921	0.9984
Decision Tree	0.021	0.9436	1.7080	1.3069	0.9981

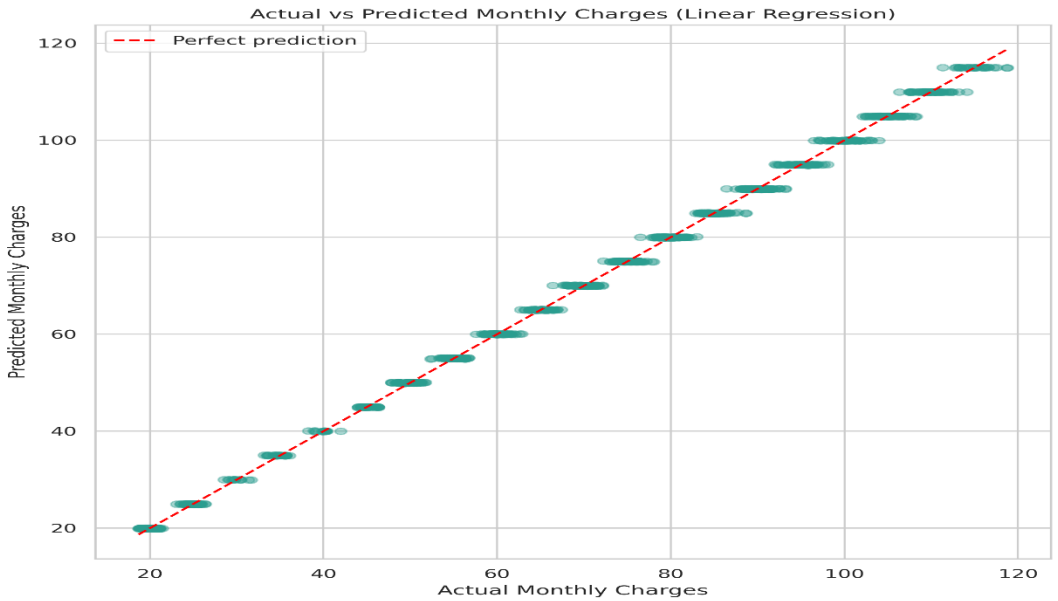


Figure 11: Actual vs predicted monthly charges (Linear Regression)

Linear Regression is the best model on every metric: on average its prediction is only about 0.79 dollars away from the real bill. The very high R^2 is not a mistake. The monthly bill of a telecom customer is built from fixed prices of the services the customer uses, so the service columns explain the bill almost completely, and a simple linear model that adds up their effects works best. The horizontal bands in the chart appear because many plans have prices that are multiples of about 5 dollars.

SECTION 9: Model Comparison

9.1 Classification Models

Table 11 compares the models by training time, evaluation metrics, advantages and limitations. They are ranked by F1 score, which is more reliable than accuracy on this imbalanced dataset.

Table 11: Comparison of the classification models

Model	Time (s)	Acc / Prec / Rec / F1 (%)	Advantages	Limitations
1. Logistic Regression	0.126	80.62 / 65.93 / 55.88 / 60.49	Simple, fast, easy to interpret, no overfitting, best F1	Only linear boundaries; recall still about 56%
2. XGBoost	0.136	79.99 / 65.44 / 52.14 / 58.04	Highest ROC-AUC, fast, handles complex patterns	Many hyperparameters; lower recall than Logistic Regression
3. SVM	9.481	79.63 / 65.37 / 49.47 / 56.32	Good precision, works well on scaled data	Slowest to train, lowest recall among top models, hard to interpret
4. Random Forest	1.770	78.85 / 62.75 / 50.00 / 55.65	Combines many trees, gives feature importance	Strong overfitting (99.8% train accuracy), slower
5. KNN	0.013	76.30 / 55.65 / 52.67 / 54.12	Very simple, almost no training time	Slow predictions, sensitive to scaling and binary columns
6. Decision Tree	0.083	74.02 / 51.09 / 50.27 / 50.67	Easy to understand, no scaling needed	Severe overfitting (gap 25.78 points), lowest ROC-AUC

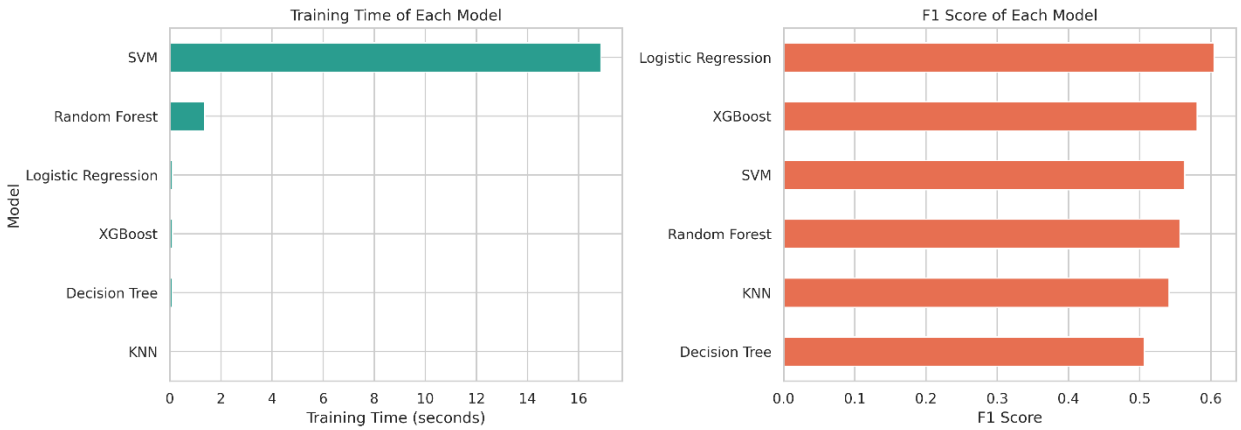


Figure 12: Training time and F1 score of each model

SVM is by far the slowest model (about 9.5 seconds), followed by Random Forest (about 1.8 seconds), while Logistic Regression trains in about 0.13 seconds and has the best F1 score. A longer training time therefore does not mean better results here.

9.2 Regression Models

Table 12: Comparison of the regression models

Model	Time (s)	MAE / RMSE / R^2	Advantages	Limitations
1. Linear Regression	0.025	0.7893 / 1.0513 / 0.9988	Best on all metrics, easy to interpret, matches additive pricing	Assumes a linear relationship
2. XGBoost	1.028	0.8193 / 1.0944 / 0.9987	Very accurate, captures non-linear effects	Needs tuning; no gain over the linear model
3. Random Forest	2.520	0.8966 / 1.1921 / 0.9984	Robust, handles non-linear effects	Slowest to train, not better than Linear Regression
4. Decision Tree	0.021	0.9436 / 1.3069 / 0.9981	Easy to understand, very fast	Lowest accuracy; depth must be limited

SECTION 10: Hyperparameter Tuning (Bonus)

Hyperparameters are settings chosen before training, such as the tree depth or the regularization strength. Four models were tuned: GridSearchCV tries every combination of a small grid (Logistic Regression, Decision Tree, Random Forest), and RandomizedSearchCV tries 20 random combinations from a large space (XGBoost). Each combination was evaluated with 5-fold stratified cross-validation on the training data only, using the F1 score for selection. The test set was not used during tuning, so the final comparison is fair. class_weight and scale_pos_weight were included so that the models can pay more attention to the churners. SVM and KNN were not tuned because SVM is slow and KNN was clearly weaker.

Table 13: Best hyperparameters found

Model	Method	Best Parameters	CV F1
Logistic Regression	GridSearchCV	C = 0.1, penalty = l1, class_weight = balanced	0.6313
Decision Tree	GridSearchCV	max_depth = 7, min_samples_leaf = 30, class_weight = balanced	0.6046
Random Forest	GridSearchCV	n_estimators = 100, max_depth = 10, min_samples_leaf = 8, class_weight = balanced	0.6397
XGBoost	RandomizedSearchCV	n_estimators = 300, max_depth = 3, learning_rate = 0.05, subsample = 0.85, colsample_bytree = 1.0, scale_pos_weight = 2.77	0.6336

All four models chose to give more weight to the churn class, and the Decision Tree and Random Forest chose limited depth and larger leaves, which reduces overfitting. The tuned models were then compared with the original models on the test set (Table 14).

Table 14: Test-set metrics before and after tuning (before → after)

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Random Forest	0.7885 → 0.7622	0.6275 → 0.5360	0.5000 → 0.7754	0.5565 → 0.6339	0.8217 → 0.8424
XGBoost	0.7999 → 0.7445	0.6544 → 0.5121	0.5214 → 0.7914	0.5804 → 0.6218	0.8442 → 0.8438
Logistic Regression	0.8062 → 0.7438	0.6593 → 0.5113	0.5588 → 0.7834	0.6049 → 0.6188	0.8422 → 0.8407
Decision Tree	0.7402 → 0.7268	0.5109 → 0.4909	0.5027 → 0.7941	0.5067 → 0.6067	0.6641 → 0.8294

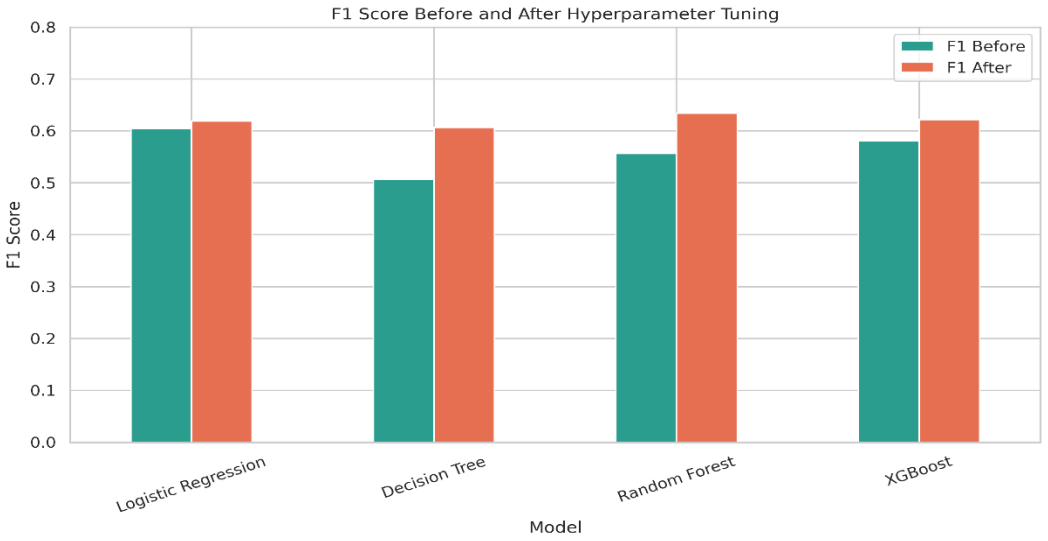


Figure 13: F1 score before and after hyperparameter tuning

Tuning improved the F1 score of all four models (Decision Tree +0.1000, Random Forest +0.0774, XGBoost +0.0414, Logistic Regression +0.0139), and the Random Forest became the best model (F1 0.6339), which agrees with the cross-validation ranking. Recall rose from about 50-56% to 77-79%, so the tuned models find about 78% of the customers who leave. The price is lower precision and accuracy, because the models now flag more customers as churners. ROC-AUC shows what really improved: for Logistic Regression and XGBoost it stayed almost the same, so the gain mostly comes from the class weights that move the decision towards "churn", while for the Random Forest (0.8217 to 0.8424) and the Decision Tree (0.6641 to 0.8294) it clearly increased because limiting depth removed most of their overfitting.

SECTION 11: Best Model Analysis

The tuned Random Forest was analyzed in more detail. On the 1,409 test customers it produced 784 true negatives, 251 false positives, 84 false negatives and 290 true positives. It finds 290 of the 374 churners (recall 77.54%) and misses 84. The default Random Forest found only 187 churners, so tuning found 103 more churners at the cost of 140 additional false alarms (251 instead of 111).

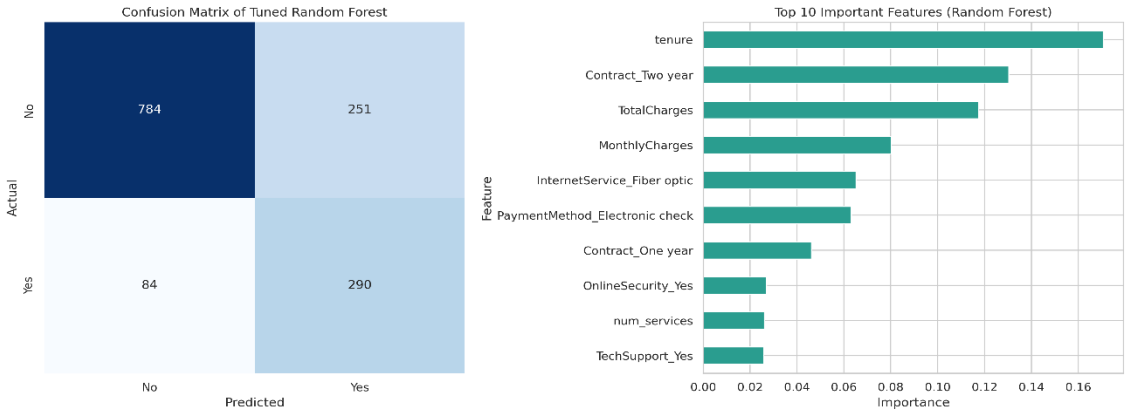


Figure 14: Confusion matrix and top 10 features of the tuned Random Forest

The most important features are tenure (0.1707), a two-year contract (0.1304) and TotalCharges (0.1176), which together account for about 42% of the importance, followed by MonthlyCharges (0.0802), fiber optic internet (0.0652) and payment by electronic check (0.0633). The engineered feature num_services is also in the top ten. To see the direction of these effects, the churn rate was calculated for each group (Table 15).

Table 15: Churn rate by key features

Feature	Group	Churn Rate
Contract	Month-to-month	42.71%
Contract	One year	11.27%
Contract	Two year	2.83%
Tenure (months)	0-12	47.44%
Tenure (months)	13-24	28.71%
Tenure (months)	25-48	20.39%
Tenure (months)	49-72	9.51%
Internet service	Fiber optic	41.89%
Internet service	DSL	18.96%
Internet service	No internet	7.40%
Payment method	Electronic check	45.29%
Payment method	Mailed check	19.11%
Payment method	Bank transfer (automatic)	16.71%
Payment method	Credit card (automatic)	15.24%

Month-to-month customers are about 15 times more likely to leave than two-year customers, and the first year is the most dangerous period (47.44% churn). Fiber optic customers churn more than twice as much as DSL customers, and electronic check users churn about three times more than customers who pay automatically. These results show an association, not proof of cause. The

company could reduce churn by focusing retention offers on new, month-to-month customers and by encouraging longer contracts and automatic payment.

SECTION 12: Conclusion

In this task, the Telco Customer Churn dataset (7,043 customers, 26.54% churn) was used to predict which customers will leave. Preprocessing fixed the hidden missing values in TotalCharges, found no duplicates, dropped the identifier column, created two new features, encoded the categorical columns and split the data 80/20 with the same churn ratio in both parts. The numerical features were scaled after the split to avoid data leakage.

Six classifiers were trained and evaluated with accuracy, precision, recall, F1 score, ROC-AUC and confusion matrices. With default settings Logistic Regression was the best (accuracy 80.62%, F1 0.6049), while the Decision Tree and Random Forest overfitted strongly. However, every model found only about half of the churners, so accuracy alone was misleading. A second task, predicting MonthlyCharges, covered the regression metrics: Linear Regression was the best (MAE 0.79, RMSE 1.05, R^2 0.9988), because the monthly bill is almost an additive sum of fixed service prices.

Hyperparameter tuning improved all four tuned models. The tuned Random Forest is recommended as the best model: it has the highest F1 score on the test set (0.6339, up from 0.5565) and in cross-validation (0.6397), and it finds 77.54% of the churners (up from 50.00%) with a ROC-AUC of 0.8424. It performed better than the others because limiting its depth and leaf size removed the overfitting of the default forest, class weights made it pay attention to the minority class, and many trees capture non-linear patterns that Logistic Regression cannot. The price is lower precision (53.60%) and accuracy (76.22%), which is acceptable because losing a customer costs more than an extra offer. Logistic Regression remains a good alternative when speed and interpretability matter most. Limitations are the single train-test split and the modest precision; future work could tune the decision threshold according to the real cost of a lost customer. Through this task I learned that avoiding data leakage, choosing the right metrics for imbalanced data and careful tuning matter as much as the choice of algorithm.

SECTION 13: GitHub Repository Link

Table 16: Project links

Item	Link
GitHub Repository	https://github.com/Amankhan987/Task_02_AI-ML_Customer_Churn_Prediction
Dataset Source (Kaggle)	https://www.kaggle.com/datasets/blastchar/telco-customer-churn

The repository contains the Jupyter notebook, the dataset, all charts and comparison tables, requirements.txt and README.md.